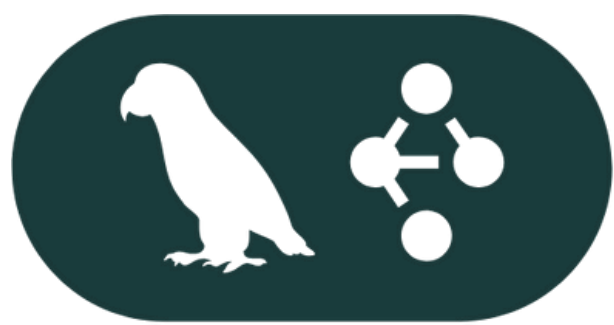




LangGraph

Explained under
1 minute!

The Problem

LangChain is great for *chaining steps together*. But once your app logic grows like branching conversations, multiple agents working together, loops, retries, conditional paths it becomes painful to manage.

Trying to force this into a linear "chain" is like writing spaghetti code with prompts.

You lose control, it's hard to debug, and things break.

Enter ***LangGraph***!

LangGraph is a library that lets you build stateful, multi-step, dynamic LLM workflows using a graph-based structure.

In simple terms:

Instead of chaining LLM calls in a straight line, LangGraph lets you design them like a flowchart with branches, loops, memory, and dynamic decision-making built-in.

You might need:

- A multi-turn assistant that loops until a task is done
- Multiple agents (planner + executor) to collaborate
- Conditional logic (if this, do that)
- Retry failed steps
- Visualize, debug, and scale LLM workflows easily

LangGraph solves all of that, with a clean structure.

What it Does

LangGraph helps you:

- Design LLM workflows as graphs (nodes + edges)
- Handle branching, loops, memory, and state
- Coordinate multiple agents or tools in one workflow
- Maintain state across steps (like a to-do list or conversation history)
- Run workflows step-by-step or asynchronously
- Visualize and debug your app logic like a proper system

How Does **LangGraph** Work?

LangGraph uses:

- **Nodes** = functions (LLM calls, tool calls, logic)
- **Edges** = transitions (what happens next based on output/state)
- **State** = memory or variables that evolve over the graph

It's built on top of LangChain and NetworkX (a Python graph library), so it feels familiar but more powerful.

You define:

1. Your state schema (what data is being passed around)
2. The graph (nodes and how they connect)
3. The logic (what to do at each node, how to move to the next)

Why Developers **Love** LangGraph

- **Visual structure:** Easier to reason about than nested chains
- **Control and flexibility:** You decide the logic, not the model
- **Modular:** Easy to reuse and extend components
- **Works with LangChain:** So your existing tools and memory still fit
- **Built for complexity:** Handles real-world app behavior like a pro

If LangChain is like writing scripts for LLMs, LangGraph is like designing full-blown systems—with clarity, structure, and power.